```
Create database OTT_movies;

use OTT_movies;

#Table Structure - Movies

CREATE TABLE movies (
   movie_id INT PRIMARY KEY,
   movie_name VARCHAR(100),
   genre VARCHAR(30),
   language VARCHAR(30),
   release_year INT,
   duration INT, -- minutes
   rating DECIMAL(3,1)
);

#Insert Data

INSERT INTO movies VALUES
(1,'Jai Bhim','Drama','Tamil',2021,164,8.8),
(2,'RRR','Action','Telugu',2022,187,8.6),
(3,'KGF Chapter 2','Action','Kannada',2022,168,8.4),
(4,'Vikram','Action','Tamil',2022,175,8.3),
(5,'Soorarai Pottru','Drama','Tamil',2020,153,8.7),
(6,'Drishyam 2','Thriller','Malayalam',2021,152,8.2),
(7,'Dangal','Sports','Hindi',2016,161,8.4),
(8,'Baahubali 2','Fantasy','Telugu',2017,171,8.2),
(9,'Andhadhun','Thriller','Hindi',2018,139,8.2),
```

```
(10,'Kantara','Drama','Kannada',2022,148,8.3),
(11,'Leo','Action','Tamil',2023,164,7.8),
(12,'Jawan','Action','Hindi',2023,169,7.9),
(13,'Pushpa','Action','Telugu',2021,179,7.6),
(14,'3 Idiots','Comedy','Hindi',2009,170,8.4),
(15,'Premam','Romance','Malayalam',2015,156,8.3);

#ACTORS TABLE

CREATE TABLE actors (
  actor_id INT PRIMARY KEY,
  actor_name VARCHAR(50),
  gender VARCHAR(10)
);

INSERT INTO actors VALUES
(1,'Surya','Male'),
(2,'Vijay','Male'),
(3,'Kamal Haasan','Male'),
(4,'Yash','Male'),
(5,'Ram Charan','Male'),
(6,'Jr NTR','Male'),
(7,'Aamir Khan','Male'),
(8,'Prithviraj','Male'),
(9,'Rajkummar Rao','Male'),
(10,'Alia Bhatt','Female');

#MOVIE_CAST TABLE (Many-to-Many)
```

```
CREATE TABLE movie_cast (
   movie_id INT,
   actor_id INT,
   role_name VARCHAR(50),
   PRIMARY KEY (movie_id, actor_id)
);

INSERT INTO movie_cast VALUES
(1,1,'Lawyer'),
(2,5,'Freedom Fighter'),
(2,6,'Revolutionary'),
(3,4,'Rocky'),
(4,3,'Agent Vikram'),
(5,1,'Pilot'),
(6,8,'Father'),
(7,7,'Wrestler'),
(8,5,'Prince'),
(9,9,'Blind Pianist'),
(10,4,'Tribal Hero'),
(11,2,'Gangster'),
(12,2,'Vigilante'),
(13,4,'Smuggler'),
(14,7,'Student'),
(15,8,'College Student');

# OTT PLATFORMS TABLE
```

```
CREATE TABLE ott_platforms (
   platform_id INT PRIMARY KEY,
   platform_name VARCHAR(30),
   launch_year INT
);

INSERT INTO ott_platforms VALUES
(1,'Netflix',2016),
(2,'Amazon Prime',2016),
(3,'Disney+ Hotstar',2019),
(4,'Zee5',2018),
(5,'Sony LIV',2020);

#MOVIE_STREAMING TABLE

CREATE TABLE movie_streaming (
   movie_id INT,
   platform_id INT,
   streaming_year INT,
   PRIMARY KEY (movie_id, platform_id)
);

INSERT INTO movie_streaming VALUES
(1,2,2021),
(2,1,2022),
(3,2,2022),
(4,3,2022),
(5,2,2020),
```

```
(6,3,2021),
(7,1,2017),
(8,1,2018),
(9,2,2019),
(10,4,2022),
(11,3,2023),
(12,1,2023),
(13,2,2021),
(14,1,2010),
(15,2,2016);

#USERS TABLE

CREATE TABLE users (
   user_id INT PRIMARY KEY,
   user_name VARCHAR(50),
   city VARCHAR(30)
);

INSERT INTO users VALUES
(1,'Arun','Chennai'),
(2,'Divya','Coimbatore'),
(3,'Rahul','Bangalore'),
(4,'Sneha','Hyderabad'),
(5,'Karthik','Mumbai');

#VIEWING HISTORY TABLE
```

```
CREATE TABLE viewing_history (
   view_id INT PRIMARY KEY,
   user_id INT,
   movie_id INT,
   watch_date DATE,
   watch_hours DECIMAL(3,1)
);

INSERT INTO viewing_history VALUES
(1,1,1,'2023-01-10',2.5),
(2,1,4,'2023-02-12',3.0),
(3,2,2,'2023-03-15',2.8),
(4,3,3,'2023-04-18',3.1),
(5,4,5,'2023-05-20',2.6),
(6,5,7,'2023-06-25',2.7),
(7,2,10,'2023-07-01',2.4),
(8,3,11,'2023-08-05',2.9);
```

# PART A – Single Table Queries (Movies / OTT / Users)

#1. Display all movies.

```
Select *
from movies;
```

#2. List movie name, genre, and rating.

```
Select movie_name, genre, rating
from movies;
```

#3. Display all action movies.

```
Select movie_name as ActionMovies
from movies
where genre = "Action";
```

#4. List movies released after 2020.

```
Select movie_name
from movies
where release_year > 2020;
```

#5. Display movies with rating above 8.5.

```
Select movie_name
from movies
where rating > 8.5;
```

#6. Show distinct movie languages.

```
Select distinct language
from movies;
```

#7. Display top 5 highest-rated movies.

```
Select movie_name, rating
from movies
order by rating desc
limit 5;
```

#8. Count total number of movies.

```
Select count(*) as TotalMovies
from movies;
```

#9. Find average movie duration.

```
Select avg(duration)
from movies;
```

#10. Display movies ordered by release year.

```
Select movie_name, release_year
from movies
order by release_year;
```

# PART B – GROUP BY & Aggregates

#11. Genre-wise movie count.

```
Select genre, count(*)
from movies
```

```
group by genre;
```

#12. Language-wise average rating.

```
Select avg(rating), language
from movies
group by language;
```

#13. Find maximum rating per genre.

```
Select Max(rating), genre
from movies
group by genre;
```

#14. Count movies released each year.

```
Select release_year, count(*)
from movies
group by release_year;
```

#15. Display genres having more than 2 movies.

```
Select  genre
from movies
group by genre
having count(*) >2;
```

#  PART C – JOIN Queries (Movies, OTT, Streaming)

#16. List movies with their OTT platforms.

```
Select m.movie_name, o.platform_name
from movies m, movie_streaming ms, ott_platforms o
where m.movie_id = ms.movie_id
and ms.platform_id = o.platform_id;
```

#17. Display movies available on Netflix.

```
Select m.movie_name, o.platform_name
from movies m, movie_streaming ms, ott_platforms o
where m.movie_id = ms.movie_id
and ms.platform_id = o.platform_id
and o.platform_name = "Netflix";
```

#18. Count movies per OTT platform.

```
Select count(m.movie_name), o.platform_name
from movies m, ott_platforms o, movie_streaming ms
where ms.movie_id = m.movie_id
and o.platform_id = ms.platform_id
group by o.platform_name;
```

#19. Find OTT platform with maximum movies.

```
Select count(m.movie_name), o.platform_name
from movies m, ott_platforms o, movie_streaming ms
```

```
where ms.movie_id = m.movie_id
and o.platform_id = ms.platform_id
group by o.platform_name
order by count(m.movie_name) desc
limit 1;
```

#20. Display movies streamed after 2021

```
Select m.movie_name, ms.streaming_year
from movies m, movie_streaming ms, ott_platforms o
where ms.movie_id = m.movie_id
and o.platform_id = ms.platform_id
and streaming_year > 2021;
```

# PART D – Actor & Cast Analysis

#21. List actors and their movies.

```
Select m.movie_name, a.actor_name
from movies m, actors a, movie_cast mc
where m.movie_id = mc.movie_id
and a.actor_id = mc.actor_id;
```

#22. Count number of movies per actor.

```
Select count(m.movie_name), a.actor_name
from movies m, actors a, movie_cast mc
```

```
where m.movie_id = mc.movie_id
and a.actor_id = mc.actor_id
group by a.actor_name;
```

#23. Actor who acted in maximum movies.

```
Select count(m.movie_name), a.actor_name
from movies m, actors a, movie_cast mc
where m.movie_id = mc.movie_id
and a.actor_id = mc.actor_id
group by a.actor_name
order by count(m.movie_name) desc
limit 1;
```

#24. List movies featuring actor ‘Yash’.

```
Select m.movie_name, a.actor_name
from movies m, actors a, movie_cast mc
where m.movie_id = mc.movie_id
and a.actor_id = mc.actor_id
and a.actor_name = "Yash";
```

#25. Find actors involved in action movies.

```
Select m.movie_name, a.actor_name
from movies m, actors a, movie_cast mc
where m.movie_id = mc.movie_id
and a.actor_id = mc.actor_id
```

```
and m.genre = "Action";
```

# PART E – User Viewing Analysis

#26. List users and movies they watched.

```
Select u.user_name, m.movie_name
from movies m, users u, viewing_history v
where m.movie_id = v.movie_id
and u.user_id = v.user_id;
```

#27. Count movies watched by each user.

```
Select count(m.Movie_name), u.user_name
from movies m, users u, viewing_history v
where m.movie_id = v.movie_id
and u.user_id = v.user_id
group by u.user_name;
```

#28. User who watched maximum movies.

```
Select count(m.Movie_name), u.user_name
from movies m, users u, viewing_history v
where m.movie_id = v.movie_id
and u.user_id = v.user_id
group by u.user_name
order by count(u.user_name) desc
limit 1;
```

#29. Most watched movie.

```
Select max(v.view_id), m.movie_name
from movies m, viewing_history v
where v.movie_id = m.movie_id
group by m.movie_name
order by max(v.view_id) desc
limit 1;
```

#30. Total watch hours per movie.

```
Select sum(v.watch_hours), m.movie_name
from movies m, viewing_history v
where v.movie_id = m.movie_id
group by m.movie_name;
```

# PART F – Subqueries & Analytics

#31. Movies with rating above average.

```
Select movie_name , rating
from movies
where rating > (Select avg(rating) from movies);
```

#32. Movies longer than average duration.

```
Select movie_name , duration
```

```
from movies
where duration > (Select avg(duration) from movies);
```

#33. OTT platforms hosting action movies.

```
Select distinct o.platform_name
from ott_platforms o, movie_streaming ms,movies m
where o.platform_id = ms.platform_id
and ms.movie_id = m.movie_id
and m.genre = 'Action';
```

#34. Users who watched movies rated above 8.5.

```
Select distinct u.user_name
from users u, movies m ,viewing_history v
where u.user_id = v.user_id
and v.movie_id = m.movie_id
and m.rating > 8.5;
```

#35. Movies not yet watched by any user.

```
Select * from movies
where movie_id not in (select movie_id from viewing_history);
```

# 🔹 PART G – Date & Reporting Queries

#36.Movies watched in 2023.

```
Select distinct m.movie_name
from movies m,viewing_history v
where m.movie_id = v.movie_id
and year(v.watch_date) = 2023;
```

#37.Users who watched movies in multiple months.

```
Select u.user_name
from users u,viewing_history v
where u.user_id = v.user_id
group by u.user_name
having count(distinct month(v.watch_date)) > 1;
```

#38.Movies streamed before release year.

```
Select m.movie_name
from movies m,movie_streaming ms
where m.movie_id = ms.movie_id
and year(ms.streaming_year) < m.release_year;
```

#39. Latest released movie.

```
Select * from movies
order by release_year desc
limit 1;
```

#40. Oldest movie in dataset.

```
Select * from movies
order by release_year asc
limit 1;
```

# 🔹 PART H – Ranking & Advanced (Optional)

#41. Rank movies by rating.

```
Select movie_name, rating,
Rank() Over (Order by rating desc) as Ranking
from movies;
```

#42. Rank actors by movie count.

```
Select a.actor_name, count(mc.movie_id),
rank() over(order by count(mc. movie_id) desc)
from actors a, movie_cast mc
where a.actor_id = mc.actor_id
group by a.actor_name;
```

#43. Platform-wise average movie rating.

```
Select o.platform_name, round(avg(m.rating),2)
from ott_platforms o,movie_streaming ms,movies m
where o.platform_id = ms.platform_id
and ms.movie_id = m.movie_id
group by o.platform_name;
```

#44.Most popular genre based on views.

```
Select m.genre, Count(v.view_id) as total_views
from movies m,viewing_history v
where m.movie_id = v.movie_id
group by m.genre
order by total_views desc
limit 1;
```

#45.City-wise user watch count.

```
Select u.city, count(v.view_id) as watch_count
from users u,viewing_history v
where u.user_id = v.user_id
group by u.city;
```

# 🔹 PART I – Case Study / Business Queries

#46.Categorize movies based on rating.

```
Select movie_name,rating,
   case
      when rating >= 8.5 then 'Excellent'
      When rating >= 8.0 then 'Good'
      When rating >= 7.5 then 'Average'
      else 'Below Average'
   end as rating_category
from movies;
```

#47. Identify binge-watch users (watch_hours > 2.5).

```
Select distinct u.user_name, v.watch_hours
from users u, viewing_history v
where u.user_id = v.user_id
and v.watch_hours > 2.5;
```

#48.Movies available on more than one platform.

```
Select m.movie_name, count(ms.platform_id)
from movies m,movie_streaming ms
where m.movie_id = ms.movie_id
group by m.movie_name
having count(ms.platform_id) > 1;
```

#49. Find platform with highest total watch hours.

```
Select o.platform_name, sum(v.watch_hours)
from ott_platforms o, movie_streaming ms,viewing_history v
where o.platform_id = ms.platform_id
and ms.movie_id = v.movie_id
group by o.platform_name
order by sum(v.watch_hours) desc
limit 1;
```

#50. Generate movie performance report.

```
Select m.movie_name, m.genre, m.rating,
count(v.view_id) as total_views,
sum(v.watch_hours) as Hours_watched
from movies m, viewing_history v
where m.movie_id = v.movie_id
group by m.movie_name, m.genre, m.rating
order by Hours_watched desc;
```